

Lecture Notes: The `java.sql.Connection` Interface

Think of the Connection as a Phone Line between your Java program and the Database. Once the line is open, you need to decide *how* you want to speak. You have three options (methods):

1. Statement Interface

- Definition: "It is used to execute static sql queries."
- Simple Explanation: Use this for queries that never change.
 - *Example:* A query like `SELECT * FROM Employees`. It is always the same. It's like a printed menu at a restaurant—you can't change the items.
- `createStatement()`

2. PreparedStatement Interface

- Definition: "It is used to execute dynamic sql queries."
- Simple Explanation: Use this for queries that represent a specific structure but have changing values.
 - *Example:* A login screen. The query structure is `SELECT * FROM Users WHERE name = ?`. The `?` (question mark) waits for the user's input. It's like a form you fill out.
- `prepareStatement(String sql)`

3. CallableStatement Interface

- Definition: "It is used to execute PL/SQL programs."
- Simple Explanation: Use this to run complex programs stored inside the database.
 - *Example:* A database function that calculates "Monthly Interest" for 1,000 customers automatically. Instead of writing the math in Java, you just "Call" the database function to do it for you.
- `prepareCall(String sql)`

Once you have created a `Statement` object (using `createStatement()`), you need to choose the correct method to run your SQL query. Java provides three specific methods depending on what kind of SQL you are writing.

1. `execute()` Method

- Signature: `public boolean execute(String sql) throws SQLException`

- Use Case: "It is suitable to execute DDL queries."
- What is DDL?
 - Full Form: Data Definition Language.
 - Meaning: These are queries that define or change the structure of the database, not the actual data inside it.
 - Examples: **CREATE**, **ALTER**, **DROP**.
- Analogy: Think of this as Construction Work. You are building the house (**CREATE**) or knocking down a wall (**ALTER**).

2. **executeUpdate()** Method

- Signature: **public int executeUpdate(String sql) throws SQLException**
- Use Case: "It is suitable to execute DML queries."
- What is DML?
 - Full Form: Data Manipulation Language.
 - Meaning: These are queries that change the data inside the tables.
 - Examples: **INSERT**, **UPDATE**, **DELETE**.
- Analogy: Think of this as Staff Work. You are adding new inventory (**INSERT**) or changing a price tag (**UPDATE**).

3. **executeQuery()** Method

- Signature: **public ResultSet executeQuery(String sql) throws SQLException**
- Use Case: "It is suitable to execute DQL queries."
- What is DQL?
 - Full Form: Data Query Language.
 - Meaning: These are queries that simply fetch or read data without changing anything.
 - Example: **SELECT**.
- Analogy: Think of this as a Customer Reading a Menu. They are just looking at what is there (**SELECT**); they aren't building the restaurant or cooking the food.

CRUD Operation	SQL Command	Java Method to Use	Why?

Create	INSERT	executeUpdate()	Because we are changing data (DML).
Read	SELECT	executeQuery()	Because we are fetching data (DQL).
Update	UPDATE	executeUpdate()	Because we are changing data (DML).
Delete	DELETE	executeUpdate()	Because we are changing data (DML).

The `java.sql.ResultSet` Interface

1. What is a ResultSet?

- Concept: When you run a `SELECT` query, the database sends back a table of data. In Java, this table is stored in an object called the `ResultSet`.
- Visual: Think of it as a spreadsheet held in memory. However, you can't see the whole spreadsheet at once. You can only see one row at a time.

2. The "Cursor" Concept

Reference: The `next()` method

- The `ResultSet` has a movable pointer called a Cursor.
- Default Position: When the object is first created, the cursor points Before the First Row (Header area). It does not point to data yet.

3. Key Methods Explained

A. `next()` – The Mover

- Signature: `public boolean next() throws SQLException`
- Role: It does two things:
 1. Moves the cursor forward to the next row.

- 2. Returns `true` if there is a row to read, or `false` if you have reached the end.
- Analogy: It's like turning the page of a book. If there is a next page, you turn it (`true`). If you are at the back cover, you stop (`false`).

B. The Getter Methods (`getInt`, `getString`, etc.) Reference: `getString(int)`, `getInt(int)`

- Once the cursor is on a row, you use these methods to pull data out of the columns.
- Important Rule: You must match the Java method to the Database data type.
 - If Column 1 is a Number (ID) -> Use `getInt(1)`
 - If Column 2 is Text (Name) -> Use `getString(2)`
 - If Column 3 is Decimal (Salary) -> Use `getFloat(3)`

C. `getMetaData()`

- Role: If you want to know about the table itself (e.g., "What are the column names?" or "How many columns are there?"), you use this method. It is "Data about Data".
- `ResultSetMetaData rm=rs.getMetaData()`

`rm.getColumnCount()`

`rm.getColumnName(int)`

The Metadata Program:

```

// We run a query to get a ResultSet
ResultSet rs = stmt.executeQuery("SELECT * FROM emp");

// Now we extract the "Data about Data" from the ResultSet
ResultSetMetaData rsmd = rs.getMetaData();

System.out.println("=== Table (ResultSet) Info ===");

// 1. How many columns are in this table?
int totalColumns = rsmd.getColumnCount();
System.out.println("Total Columns: " + totalColumns);

// 2. What are the names and types of these columns?
System.out.println("-----");
System.out.println("Col Index | Column Name   | Data Type");
System.out.println("-----");

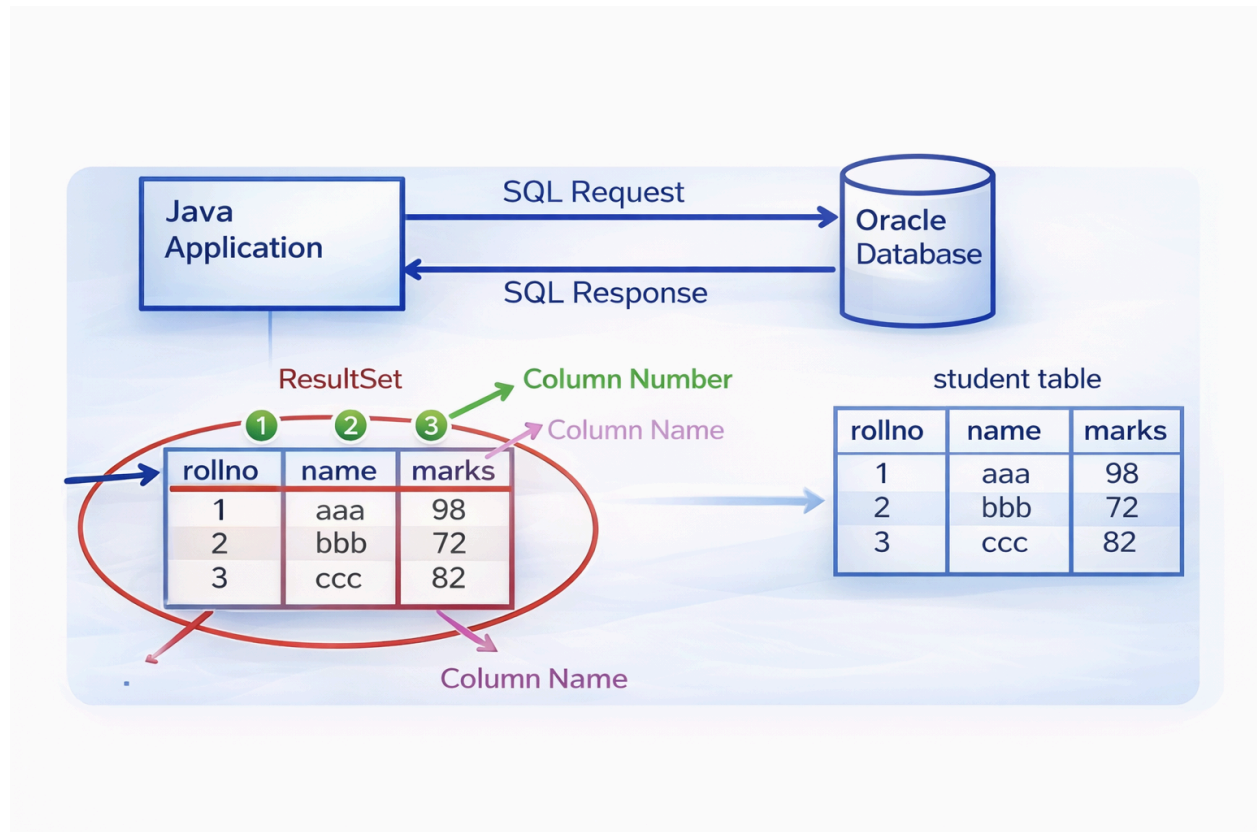
// Loop from 1 to Total Columns (Remember: JDBC starts at 1, not 0)
for(int i = 1; i <= totalColumns; i++) {
    System.out.println(
        "    " + i + "      | " +
        rsmd.getColumnName(i) + "      | " +
        rsmd.getColumnTypeName(i)
    );
}
System.out.println("-----");

con.close();

} catch (Exception e) {
    e.printStackTrace();
}

}

```



1. The CREATE Program (Insert Data)

```

import java.sql.*;

public class InsertDemo {
    public static void main(String[] args) {
        try {
            // Step 1: Load the Driver
            Class.forName("oracle.jdbc.driver.OracleDriver");

            // Step 2: Establish the Connection (The Phone Line)
            Connection con = DriverManager.getConnection(
                "jdbc:oracle:thin:@localhost:1521:xe", "system", "password");

            // Step 3: Create the Statement (The Vehicle)
            Statement stmt = con.createStatement();

            // Step 4: Execute the Query (The Action)
            // Query: Insert employee ID 101, Name 'Ravi', Salary 5000
            String sql = "INSERT INTO emp VALUES(101, 'Ravi', 5000)";

            // We use executeUpdate because this is DML (Data Manipulation)
            int rowsAffected = stmt.executeUpdate(sql);

            System.out.println("Success! Rows inserted: " + rowsAffected);

            // Step 5: Close the Connection
            con.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

2. The READ Program (Select Data)

```

import java.sql.*;

public class SelectDemo {
    public static void main(String[] args) {
        try {
            // Steps 1 & 2 are the same...
            Class.forName("oracle.jdbc.driver.OracleDriver");
            Connection con = DriverManager.getConnection(
                "jdbc:oracle:thin:@localhost:1521:xe", "system", "password");

            // Step 3: Create Statement
            Statement stmt = con.createStatement();

            // Step 4: Execute the Query
            String sql = "SELECT * FROM emp";

            // We use executeQuery because this is DQL (Data Query Language)
            // It returns a 'ResultSet' which holds the table data
            ResultSet rs = stmt.executeQuery(sql);

            // Iterate through the ResultSet (Concept: A Cursor)
            while(rs.next()) {
                // Get data from columns 1, 2, and 3
                System.out.println(rs.getInt(1) + " | " + rs.getString(2) + " | " + rs.getString(3));
            }

            con.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

3. The UPDATE Program

```

import java.sql.*;

public class UpdateDemo {
    public static void main(String[] args) {
        try {
            Class.forName("oracle.jdbc.driver.OracleDriver");
            Connection con = DriverManager.getConnection(
                "jdbc:oracle:thin:@localhost:1521:xe", "system", "password");
            Statement stmt = con.createStatement();

            // SQL: Update salary for ID 101
            String sql = "UPDATE emp SET salary = 7000 WHERE id = 101";

            // Use executeUpdate (Returns the number of rows updated)
            int rowsAffected = stmt.executeUpdate(sql);

            // Teacher's Tip: Always check if rows > 0 to know if it worked!
            if(rowsAffected > 0) {
                System.out.println("Update Successful! Salary changed.");
            } else {
                System.out.println("Update Failed. ID 101 not found.");
            }

            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

4. The DELETE Program

```

import java.sql.*;

public class DeleteDemo {
    public static void main(String[] args) {
        try {
            Class.forName("oracle.jdbc.driver.OracleDriver");
            Connection con = DriverManager.getConnection(
                "jdbc:oracle:thin:@localhost:1521:xe", "system", "password");
            Statement stmt = con.createStatement();

            // SQL: Delete employee with ID 101
            String sql = "DELETE FROM emp WHERE id = 101";

            // Use executeUpdate
            int rowsAffected = stmt.executeUpdate(sql);

            if(rowsAffected > 0) {
                System.out.println("Delete Successful! Employee removed.");
            } else {
                System.out.println("Delete Failed. ID 101 not found.");
            }

            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Dynamic SQL Queries:

1. insert into student values(?, ?, ?);
2. update student set marks=? where rollno=?;
3. delete from student where rollno=?;
4. select * from student where rollno=?;

In the context of database programming (specifically JDBC in Java), these are known as Parameterized Queries or Prepared Statements.

The question marks (?) function as positional parameters (or placeholders). Instead of hardcoding values directly into the query string, the values are supplied at runtime.

Key Characteristics:

- **Runtime Binding:** The database engine compiles the SQL statement structure first, and the specific data (like a student's name or roll number) is inserted later.
- **1-based Indexing:** When setting these values in code, the first ? corresponds to index 1, the second to index 2, and so on.

Why use this approach?

1. **Security (Prevents SQL Injection):** This is the most critical advantage. By using placeholders, the database treats user input strictly as data, not as executable code. This prevents attackers from manipulating the query logic (e.g., entering ' OR '1'='1 to bypass logins).
2. **Performance:** The database parses, compiles, and optimizes the query plan only once. If you execute the same query multiple times with different values (e.g., inserting 100 students), the database reuses the execution plan, which is significantly faster than recompiling a new query string every time.

```

import java.sql.*;
import java.util.Scanner;

public class InsertDemo {
    public static void main(String[] args) {

        // 1. Connection Details
        String url = "jdbc:mysql://localhost:3306/your_database_name";
        String user = "root";
        String pass = "password";

        try {
            // 2. Establish Connection
            Connection con = DriverManager.getConnection(url, user, pass);
            Scanner sc = new Scanner(System.in);

            // 3. Take User Input
            System.out.println("Enter Roll Number:");
            int roll = sc.nextInt();

            System.out.println("Enter Name:");
            String name = sc.next();

            System.out.println("Enter Marks:");
            double marks = sc.nextDouble();

            // 4. Create Dynamic Query (The '?' are placeholders)
            String query = "insert into student values(?, ?, ?)";
            PreparedStatement ps = con.prepareStatement(query);

            // 5. Fill the placeholders with user data
            ps.setInt(1, roll); // Sets the 1st ?
            ps.setString(2, name); // Sets the 2nd ?
            ps.setDouble(3, marks); // Sets the 3rd ?

            // 6. Execute
            int i = ps.executeUpdate();
            System.out.println(i + " row inserted successfully.");

            con.close();

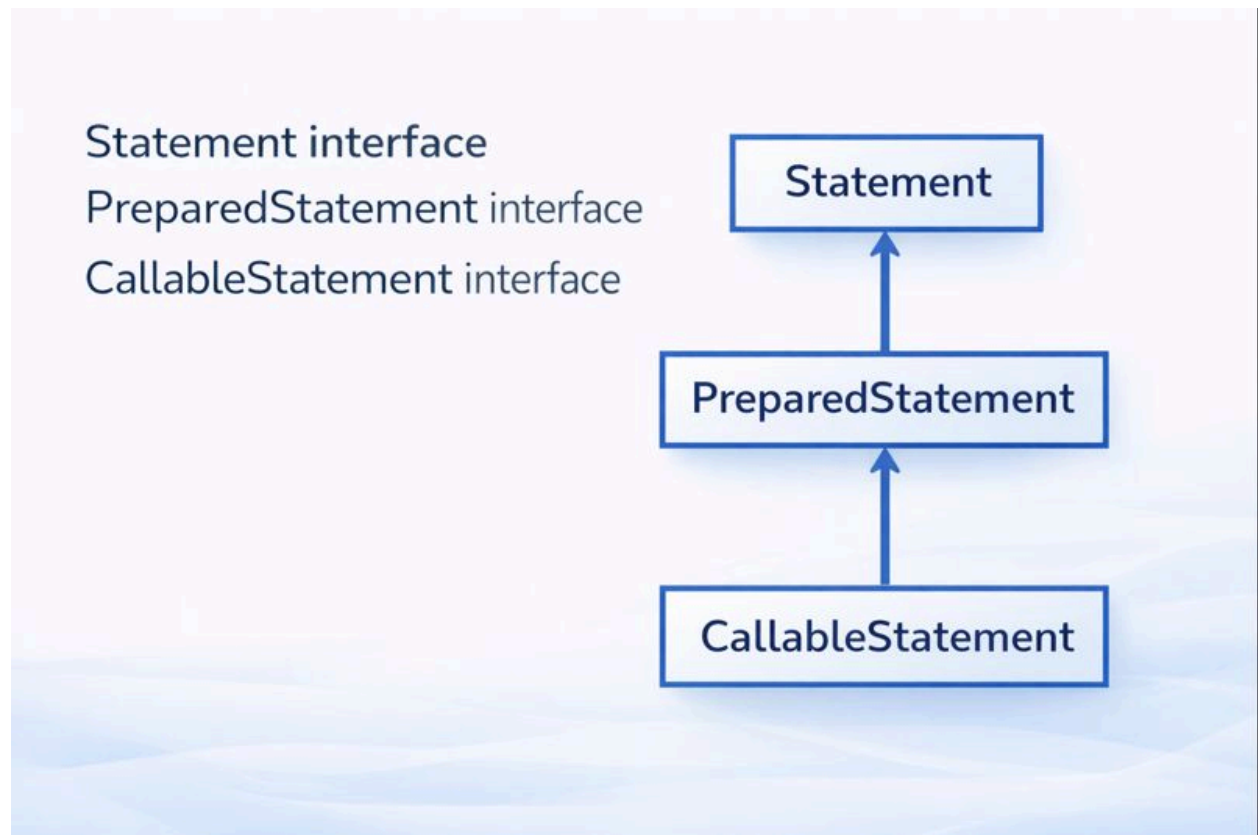
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Executing the Query

The bottom section of your slide lists the methods to actually run the SQL once the data is filled.

- **executeUpdate()**
 - Use for: **INSERT**, **UPDATE**, **DELETE** operations.
 - Returns: An **int** representing the number of rows affected (e.g., "1 row inserted").
 - *Note in your image:* It returns **int** because it counts the changes.
- **executeQuery()**
 - Use for: **SELECT** operations.
 - Returns: A **ResultSet** object containing the data fetched from the database.
- **execute()**
 - Use for: Any SQL statement (generic).
 - Returns: A **boolean**. Returns **true** if the result is a **ResultSet** (like a select), and **false** if it is an update count.



CallableStatement: Extends **PreparedStatement**. This is used for calling Stored Procedures and functions stored inside the database (a more advanced topic).